

Validação de Casos de Uso

Do comportamento esperado à interação entre objetos



Prof. Dr. João Choma

Universidade Estadual de Maringá — DIN/CTC

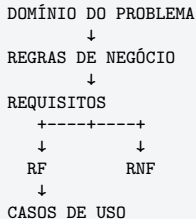
github.com/JoaoChoma

Análise e Projeto de Software ■ Diagrama de Sequência



Onde estamos?

Até aqui, trabalhamos com:



Já conseguimos responder:

- quais regras governam o domínio;
- o que o software deve fazer;
- quem utiliza as funcionalidades;
- quais objetivos os atores desejam alcançar;
- como ocorre a interação entre ator e sistema.

Agora precisamos avançar.



A pergunta desta aula

Ter um Caso de Uso escrito significa que ele está correto?

Não necessariamente.

Um Caso de Uso pode:

- esquecer regras de negócio;
- não cobrir requisitos;
- possuir passos ambíguos;
- ignorar situações de erro;
- não deixar claro quem executa cada ação;
- descrever interface em vez de comportamento;
- possuir fluxos incompletos;
- terminar sem alcançar o objetivo do ator.

Portanto:

Antes de projetar a solução, precisamos validar a especificação do comportamento.



Ao final da aula, o estudante deverá ser capaz de:

- 1 avaliar se uma especificação de Caso de Uso está correta e completa;
- 2 utilizar um checklist para revisão;
- 3 relacionar Caso de Uso, requisitos e regras de negócio;
- 4 identificar inconsistências em fluxos;
- 5 compreender a finalidade do Diagrama de Sequência;
- 6 transformar um cenário de Caso de Uso em interações;
- 7 construir um Diagrama de Sequência coerente com a especificação.

Parte I — Validando Casos de Uso



O que significa um Caso de Uso estar correto?

Não significa apenas estar:

- bem escrito;
- formatado;
- bonito;
- detalhado.

Um Caso de Uso deve ser **coerente com o domínio e com os requisitos**.

Podemos analisar quatro dimensões:

CORREÇÃO
COMPLETUDE
CONSISTÊNCIA
RASTREABILIDADE



1. Correção

Pergunta principal:

O Caso de Uso representa corretamente o comportamento esperado do sistema?

Devemos verificar:

- ator correto;
- objetivo correto;
- ações coerentes;
- resposta adequada do sistema;
- regras de negócio respeitadas;
- resultado final esperado.



2. Completude

Pergunta:

Existe alguma informação importante faltando?

Verificar:

- pré-condições;
- gatilho;
- fluxo principal;
- fluxos alternativos;
- exceções;
- pós-condições;
- regras relacionadas;
- requisitos relacionados.

Um fluxo principal correto pode continuar sendo **incompleto**.



3. Consistência

Pergunta:

As partes da especificação não se contradizem?

Exemplo:

RN05:
Clientes bloqueados não podem realizar pedidos.

Mas o Caso de Uso diz:

3. Sistema identifica que o cliente está bloqueado.
4. Sistema registra normalmente o pedido.

Temos uma inconsistência.

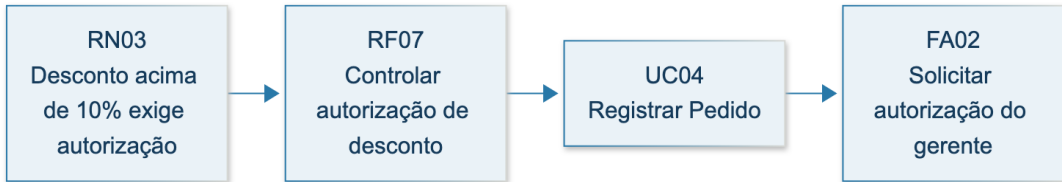


4. Rastreabilidade

Pergunta:

Conseguimos descobrir por que determinado comportamento existe?

Exemplo:



Cada comportamento importante deve possuir uma justificativa.



Testar um Caso de Uso

Aqui, "testar" significa realizar uma **revisão sistemática da especificação**.

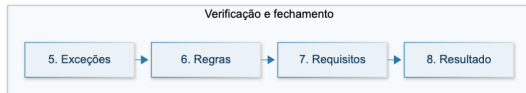
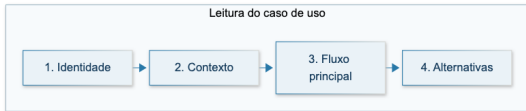
Não estamos ainda executando o software.

Estamos perguntando:

Se este documento fosse entregue para desenvolvimento e teste, haveria informação suficiente para compreender o comportamento esperado?



Podemos revisar o Caso de Uso em camadas:





Verifique:

- possui identificador?
- possui nome?
- o nome representa um objetivo?
- existe ator principal?
- existem atores secundários?
- o objetivo está claramente declarado?

Exemplo adequado:

UC04 - Registrar Pedido

Ator principal: Vendedor

Objetivo: Registrar um pedido para um cliente.



Verifique:

- quais condições precisam existir antes?
- qual evento inicia o caso de uso?
- qual resultado deve existir ao final?

Pré-condição:

Vendedor autenticado.

Gatilho:

Vendedor solicita o registro de um novo pedido.

Pós-condição:

Pedido registrado com identificador único.



Para cada passo, pergunte:

Quem executa a ação?

A ação é observável?

A resposta do sistema está clara?

O próximo passo é consequência lógica do anterior?

Uma boa especificação alterna claramente:

ATOR → SISTEMA → ATOR → SISTEMA

quando a interação exigir isso.



1. Abrir tela.
2. Digitar os dados.
3. Salvar.
4. Processar.
5. Finalizar.

Problemas:

- quem abre?
- qual tela?
- quais dados?
- quem processa?
- o que significa finalizar?
- qual resultado é produzido?
- quais regras são verificadas?



1. Vendedor solicita o registro de um novo pedido.
2. Sistema solicita a identificação do cliente.
3. Vendedor informa o cliente.
4. Sistema valida a situação cadastral do cliente.
5. Sistema apresenta os dados do cliente.
6. Vendedor adiciona os produtos desejados.
7. Sistema verifica a disponibilidade dos produtos.
8. Sistema calcula o valor total.
9. Vendedor confirma o pedido.
10. Sistema registra o pedido.
11. Sistema informa o número do pedido.

Agora existe comportamento observável.



Cada passo deve responder claramente:

```
QUEM?  
+  
FAZ O QUÊ?  
+  
SOBRE O QUÊ?
```

Exemplo:

```
Sistema + valida + situação do cliente
```

Evite:

```
Sistema processa.  
  
Sistema verifica.  
  
Sistema continua.
```

Verifica **o** quê?

Processa **o** quê?



Pergunte em cada passo do fluxo principal:

Existe outra possibilidade válida?

Exemplo:

7. Sistema verifica a disponibilidade do produto.

Possibilidades:

estoque suficiente
estoque insuficiente
estoque parcial
produto indisponível

Se alguma dessas situações for relevante para o negócio, ela precisa ser considerada.



Pergunte:

O que pode impedir a conclusão normal do Caso de Uso?

Exemplos:

- cliente bloqueado;
- dados inválidos;
- pagamento recusado;
- produto indisponível;
- autorização negada;
- operação concorrente;
- serviço externo indisponível.

Nem toda falha técnica precisa aparecer.

Devemos documentar principalmente as exceções **relevantes para o comportamento esperado**.



Para cada regra relacionada, pergunte:

Em qual passo essa regra é aplicada?

Exemplo:

RN03:
Desconto superior a 10% exige autorização do gerente.

No Caso de Uso:

8. Sistema calcula o valor do pedido.

FA03 - Desconto superior a 10%

1. Sistema identifica desconto superior a 10%.
2. Sistema solicita autorização do gerente.
3. Gerente informa sua decisão.
4. Sistema registra a decisão.

A regra deixou de ser apenas texto.

Ela passou a fazer parte do comportamento.



Pergunte:

Quais requisitos este Caso de Uso realiza?

Exemplo:

UC04 - Registrar Pedido

RF04 - Permitir registrar pedidos.

RF05 - Permitir adicionar produtos.

RF06 - Calcular total automaticamente.

RF07 - Controlar descontos.

RF08 - Validar disponibilidade em estoque.

Um Caso de Uso pode realizar vários requisitos.



Ao final:

O objetivo do ator foi alcançado?

Exemplo:

Objetivo:

Registrar um pedido.

Pós-condição:

Pedido registrado com identificador único.

Existe coerência.

Se o fluxo termina apenas com:

Sistema apresenta a tela.

provavelmente o objetivo ainda não foi demonstrado.



Estrutura básica

- O Caso de Uso possui ID?
- O nome representa um objetivo?
- O ator principal está identificado?
- Os atores secundários estão identificados?
- O objetivo está explícito?
- O gatilho está definido?
- As pré-condições estão claras?
- As pós-condições estão claras?



Checklist — Fluxo Principal

- Existe um fluxo principal completo?
- Cada passo identifica quem executa a ação?
- Os passos possuem ordem lógica?
- As ações do ator são claras?
- As respostas do sistema são claras?
- Não existem passos vagos?
- O fluxo evita detalhes desnecessários de interface?
- O fluxo termina atingindo o objetivo?



Checklist — Alternativas e Exceções

Foram identificadas decisões importantes?

Existem fluxos alternativos quando necessário?

Existem exceções relevantes?

Cada fluxo indica onde se inicia?

Está claro onde o fluxo retorna?

Está claro quando o Caso de Uso termina?

Situações inválidas possuem comportamento definido?



Checklist — Regras e Requisitos

As regras de negócio relacionadas estão identificadas?

As regras aparecem nos pontos corretos do fluxo?

Os requisitos funcionais estão relacionados?

Os RNFs relevantes foram considerados?

Não existem comportamentos sem justificativa?

Existe rastreabilidade entre $RN \rightarrow RF \rightarrow UC$?



Checklist — Qualidade

Existe ambiguidade?

Existem termos diferentes para a mesma coisa?

Existem passos técnicos demais?

Existem decisões escondidas?

O Caso de Uso poderia ser compreendido por outra equipe?

Seria possível criar testes a partir dele?

Seria possível projetar a solução a partir dele?



Para cada passo, aplique:

QUEM?
O QUÊ?
POR QUÊ?
E SE NÃO DER CERTO?
QUAL REGRA SE APLICA?
O QUE ACONTECE DEPOIS?

Se uma dessas perguntas revelar uma decisão importante não documentada:

a especificação provavelmente precisa ser melhorada.



UC04 — Registrar Pedido

Objetivo: permitir que um vendedor registre um pedido para um cliente.

Ator principal: Vendedor.

Ator secundário: Gerente.

Gatilho: vendedor solicita a criação de um novo pedido.

Pré-condições:

- vendedor autenticado;
- cliente previamente cadastrado.

Pós-condição de sucesso:

- pedido registrado com identificador único.



RN01 - O pedido deve estar associado a um cliente ativo.

RN02 - Produto sem estoque não pode ser adicionado.

RN03 - Desconto superior a 10% exige autorização do gerente.

RN04 - O valor total corresponde à soma dos subtotais dos itens menos os descontos autorizados.



RF04 - Permitir registrar pedidos.

RF05 - Permitir adicionar itens ao pedido.

RF06 - Calcular automaticamente o total.

RF07 - Controlar autorização de descontos.

RF08 - Validar disponibilidade dos produtos.



1. Vendedor solicita a criação de um pedido.
2. Sistema solicita a identificação do cliente.
3. Vendedor informa o cliente.
4. Sistema verifica se o cliente está ativo.
5. Sistema apresenta os dados do cliente.
6. Vendedor informa um produto e a quantidade.
7. Sistema verifica a disponibilidade do produto.
8. Sistema adiciona o item ao pedido.
9. Sistema calcula o subtotal e o total do pedido.
10. Vendedor confirma o pedido.
11. Sistema registra o pedido.
12. Sistema apresenta o identificador do pedido.



FA01 — Adicionar outro produto

No passo 9:

1. Vendedor solicita a inclusão de outro produto.
2. Fluxo retorna ao passo 6.

FA02 — Aplicar desconto de até 10%

Antes do passo 10:

1. Vendedor informa o desconto.
2. Sistema valida o percentual.
3. Sistema aplica o desconto.
4. Sistema recalcula o total.
5. Fluxo retorna ao passo 10.



FA03 — Desconto superior a 10%

1. Vendedor informa desconto superior a 10%.
2. Sistema identifica necessidade de autorização.
3. Sistema solicita autorização ao gerente.
4. Gerente autoriza o desconto.
5. Sistema registra a autorização.
6. Sistema aplica o desconto.
7. Sistema recalcula o total.
8. Fluxo retorna ao passo 10.

Relacionado diretamente a:

RN03 + RF07



FE01 — Cliente bloqueado

No passo 4:

1. Sistema identifica que o cliente está bloqueado.
2. Sistema informa que pedidos não podem ser registrados.
3. Caso de Uso é encerrado sem registrar o pedido.

FE02 — Estoque insuficiente

No passo 7:

1. Sistema identifica estoque insuficiente.
2. Sistema informa a indisponibilidade.
3. Produto não é incluído.
4. Fluxo retorna ao passo 6.



Podemos considerar o UC completo?

Não porque ele tem muitas linhas.

Mas porque conseguimos verificar:

```
ATOR
↓
OBJETIVO
↓
PRÉ-CONDIÇÕES
↓
FLUXO PRINCIPAL
↓
ALTERNATIVAS
↓
EXCEÇÕES
↓
REGRAS
↓
REQUISITOS
↓
PÓS-CONDIÇÃO
```

E existe coerência entre esses elementos.



Pergunte ao desenvolvedor:

Você consegue compreender o comportamento que precisa implementar?

Pergunte ao testador:

Você consegue derivar cenários de teste?

Pergunte ao responsável pelo negócio:

Esse comportamento representa corretamente o processo?

Se as respostas divergirem, existe algo a revisar.



Do UC04 podemos obter:

CT01 - Registrar pedido válido.

CT02 - Tentar registrar para cliente bloqueado.

CT03 - Adicionar produto disponível.

CT04 - Adicionar produto sem estoque.

CT05 - Aplicar desconto de 5%.

CT06 - Aplicar desconto de 15% autorizado.

CT07 - Aplicar desconto de 15% não autorizado.

Casos de Uso bem especificados também melhoram a testabilidade.



O Caso de Uso descreve:

ATOR <-> SISTEMA

Exemplo:

Vendedor → Sistema:
"registrar pedido"

Sistema → Vendedor:
"pedido registrado"

Mas...

Como o sistema realiza internamente esse comportamento?



Quando o ator solicita:

Registrar Pedido

o software precisa coordenar:

- interface;
- objetos;
- serviços;
- regras;
- persistência;
- integrações.

Precisamos representar **como os participantes colaboram ao longo do tempo.**

É aqui que entra o:

Parte II — Diagrama de Sequência



Por que o Diagrama de Sequência existe?

O Caso de Uso apresenta principalmente a perspectiva externa:

```
ATOR
↓
SISTEMA
```

O Diagrama de Sequência permite detalhar uma realização do comportamento:

```
ATOR
↓
INTERFACE
↓
CONTROLE / SERVIÇO
↓
OBJETOS DO DOMÍNIO
↓
REPOSITÓRIO / PERSISTÊNCIA
```

Ele ajuda a representar **quem interage com quem e em qual ordem.**



A pergunta respondida pelo Diagrama de Sequência

Quais participantes colaboram para realizar determinado cenário e em qual sequência as mensagens são trocadas?

Ele enfatiza:

TEMPO
+
ORDEM DAS MENSAGENS
+
RESPONSABILIDADES



Caso de Uso

O QUE deve acontecer?

Diagrama de Sequência

COMO os participantes colaboram
para realizar aquele comportamento?

O diagrama não substitui o Caso de Uso.
Ele **refina um cenário** do comportamento.



Por que elaborar?

Um Diagrama de Sequência ajuda a:

- entender a dinâmica da solução;
- distribuir responsabilidades;
- identificar operações necessárias;
- descobrir objetos e serviços;
- validar interfaces entre componentes;
- discutir decisões de projeto;
- detectar responsabilidades mal posicionadas;
- preparar implementação e testes.

O que ele deve expressar?



Principalmente:

QUEM PARTICIPA?



QUEM ENVIA UMA MENSAGEM?



QUEM RECEBE?



EM QUAL ORDEM?



QUAL RESPOSTA É PRODUZIDA?



EXISTEM CONDIÇÕES OU REPETIÇÕES?



Um Diagrama de Sequência normalmente possui:

- ① atores;
- ② participantes;
- ③ linhas de vida;
- ④ mensagens;
- ⑤ ativações;
- ⑥ retornos;
- ⑦ fragmentos combinados;
- ⑧ criação ou destruição de objetos, quando relevante.



Exemplo:

```
Vendedor  
TelaPedido  
PedidoController  
PedidoService  
EstoqueService  
PedidoRepository
```

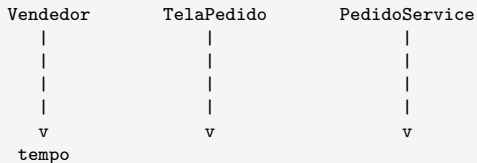
Cada participante possui uma responsabilidade.

O diagrama não deve simplesmente listar todas as classes existentes.

Inclua aquelas que participam do cenário modelado.



Cada participante possui uma linha de vida:



O tempo normalmente avança:

DE CIMA
↓
PARA BAIXO



Uma mensagem representa uma comunicação entre participantes.

Exemplo:

```
Vendedor → TelaPedido:  
confirmarPedido()
```

```
TelaPedido → PedidoController:  
registrarPedido(dados)
```

```
PedidoController → PedidoService:  
registrar(dados)
```

A ordem vertical mostra a sequência das interações.



Uma mensagem normalmente representa:

- solicitação de operação;
- chamada de método;
- comando;
- consulta;
- evento;
- comunicação entre componentes.

Exemplo:

```
PedidoService → EstoqueService:  
verificarDisponibilidade(itens)
```

Isso também revela uma responsabilidade.



Podemos representar o resultado:

```
EstoqueService → PedidoService:  
disponível
```

```
PedidoRepository → PedidoService:  
pedidoSalvo
```

Nem todo retorno trivial precisa aparecer.

Mostre retornos quando eles **ajudarem a compreender o cenário**.



A ativação representa o período no qual um participante está executando uma operação.

Conceitualmente:



Ela ajuda a visualizar quem está executando determinada responsabilidade.



São importantes para representar comportamento condicional ou repetitivo.

Os mais úteis inicialmente:

```
alt  
opt  
loop
```



Representa caminhos mutuamente exclusivos.

Exemplo:

```
alt [estoque disponível]
    adicionar item
else [estoque insuficiente]
    informar indisponibilidade
```

É semelhante a uma decisão:

```
if / else
```



Representa um comportamento executado somente se determinada condição for verdadeira.

```
opt [desconto informado]
```

```
    validar desconto
```

É semelhante a:

```
if
```

sem else.



Representa comportamento repetitivo.

Exemplo:

```
loop [para cada item]
    verificar estoque
    calcular subtotal
```

É útil quando o Caso de Uso possui repetição.



Não devemos inventá-lo sem referência.

Uma origem natural é:

```
CASO DE USO
↓
CENÁRIO
↓
PASSOS
↓
RESPONSABILIDADES
↓
PARTICIPANTES
↓
MENSAGENS
↓
DIAGRAMA DE SEQUÊNCIA
```



Qual cenário diagramar?

Um Caso de Uso pode possuir:

```
Fluxo Principal  
FA01  
FA02  
FA03  
FE01  
FE02  
...
```

Não é obrigatório colocar tudo em um único diagrama.

Podemos:

- ① criar um diagrama para o fluxo principal;
- ② incorporar alternativas relevantes;
- ③ criar diagramas separados para cenários complexos.

O objetivo é **comunicação**, não produzir um diagrama gigantesco.



Vamos utilizar:

UC04 - Registrar Pedido

Primeiro, selecione o cenário:

FLUXO PRINCIPAL

Depois analise cada passo.



Caso de Uso:

1. Vendedor solicita a criação de um pedido.
2. Sistema solicita a identificação do cliente.
3. Vendedor informa o cliente.

No projeto, podemos imaginar:

Vendedor
↓
TelaPedido

Interações iniciais:

Vendedor → TelaPedido:
novoPedido()

Vendedor → TelaPedido:
selecionarCliente(clienteId)



Caso de Uso:

4. Sistema verifica se o cliente está ativo.

Pergunta de projeto:

Quem deve possuir essa responsabilidade?

Uma possibilidade:

```
TelaPedido  
  ↓  
PedidoController  
  ↓  
ClienteService
```

Mensagem:

```
PedidoController → ClienteService:  
validarCliente(clienteId)
```



Caso de Uso:

6. Vendedor informa produto e quantidade.

7. Sistema verifica disponibilidade.

Possível colaboração:

```
Vendedor  
↓  
TelaPedido  
↓  
PedidoController  
↓  
EstoqueService
```

Mensagem:

```
PedidoController → EstoqueService:  
verificarDisponibilidade(produto, quantidade)
```



Caso de Uso:

8. Sistema adiciona o item.

9. Sistema calcula subtotal e total.

Agora aparece o domínio:

Pedido
ItemPedido

Uma possibilidade:

PedidoService → Pedido:
adicionarItem(produto, quantidade)

Pedido → ItemPedido:
criar(...)

Pedido → Pedido:
calcularTotal()



Passos 10 a 12

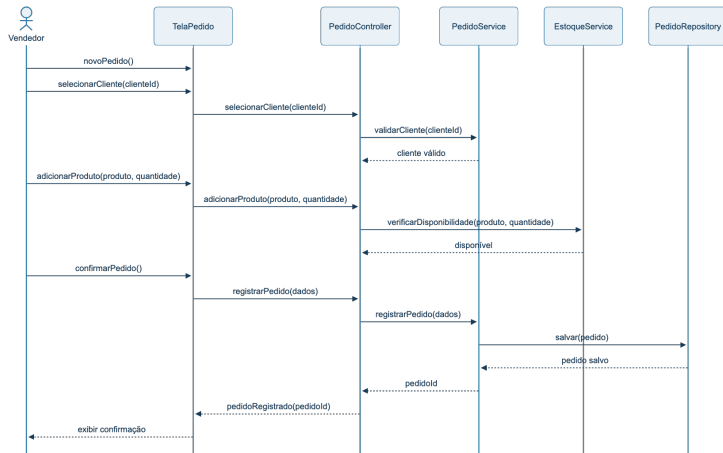
Caso de Uso:

10. Vendedor confirma o pedido.
11. Sistema registra o pedido.
12. Sistema apresenta o identificador.

Possível sequência:

```
Vendedor → TelaPedido:  
confirmar()  
  
TelaPedido → PedidoController:  
registrarPedido()  
  
PedidoController → PedidoService:  
registrarPedido()  
  
PedidoService → PedidoRepository:  
salvar(pedido)  
  
PedidoRepository → PedidoService:  
pedidoSalvo  
  
PedidoService → PedidoController:  
pedidoId  
  
PedidoController → TelaPedido:  
pedidoRegistrado(pedidoId)
```

Primeira versão do Diagrama





O diagrama revelou algo

Ao construir a sequência, começamos a perguntar:

- quem valida o cliente?
- quem verifica estoque?
- quem calcula o total?
- quem cria o pedido?
- quem persiste?
- quem coordena a operação?

Essas perguntas são importantes porque:

*estamos saindo da **análise do comportamento** e entrando no **projeto da solução**.*



No Caso de Uso:

Sistema verifica estoque.

Isso pode ser suficiente.

No Diagrama de Sequência de projeto:

```
PedidoController
    ↓
EstoqueService
    ↓
ProdutoRepository
```

começamos a decidir **como estruturar internamente a solução**.

Portanto:

o Diagrama de Sequência pode evoluir em nível de detalhe conforme o projeto evolui.



Podemos pensar em três tipos de responsabilidade:

BOUNDARY
CONTROLLER
ENTITY

ou, em arquiteturas atuais:

INTERFACE
CONTROLE / APLICAÇÃO
DOMÍNIO
INFRAESTRUTURA

Isso ajuda a evitar que um único objeto faça tudo.



Responsabilidade:

interação com atores ou sistemas externos.

Exemplos:

TelaPedido
PedidoAPI
PedidoView

Não deveria concentrar regras complexas do negócio.



Responsabilidade:

coordenar o Caso de Uso.

Exemplos:

```
PedidoController  
RegistrarPedidoUseCase  
PedidoService
```

Pode coordenar:

```
validar  
consultar  
criar  
persistir  
responder
```



Representa conceitos importantes do negócio.

Exemplos:

```
Pedido  
ItemPedido  
Cliente  
Produto
```

Pode concentrar regras que pertencem ao próprio domínio.

Exemplo:

```
pedido.adicionarItem(...)  
pedido.calcularTotal()
```



Responsável por detalhes externos:

```
PedidoRepository  
ProdutoRepository  
GatewayPagamento  
EmailService
```

Exemplo:

```
PedidoRepository.salvar(pedido)
```



Isso não é uma regra universal.

É uma forma útil de **pensar distribuição de responsabilidades**.



Regra:

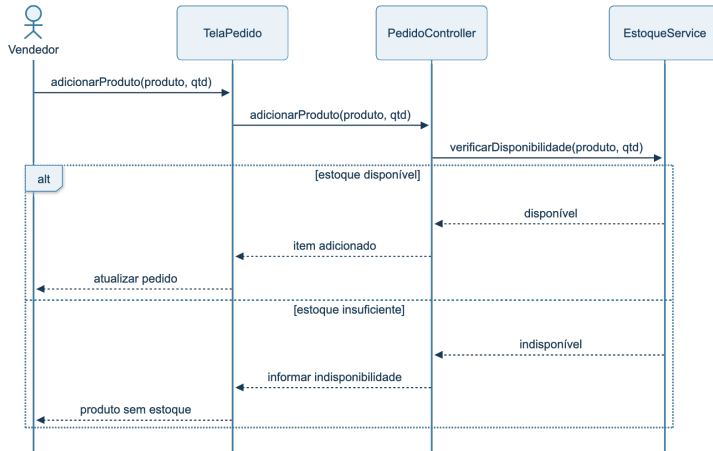
RN02 - Produto sem estoque não pode ser adicionado.

Caso de Uso:

FE02 - Estoque insuficiente

No Diagrama de Sequência:

```
alt [estoque disponível]
    adicionar item
else [estoque insuficiente]
    informar indisponibilidade
```





```
RN02
Produto sem estoque não pode ser adicionado
    ↓
RF08
Validar disponibilidade
    ↓
UC04 / FE02
Estoque insuficiente
    ↓
DIAGRAMA DE SEQUÊNCIA
alt [estoque disponível / insuficiente]
    ↓
IMPLEMENTAÇÃO
    ↓
TESTE
```

O diagrama não surge isoladamente.



Caso de Uso:

Vendedor pode adicionar vários produtos.

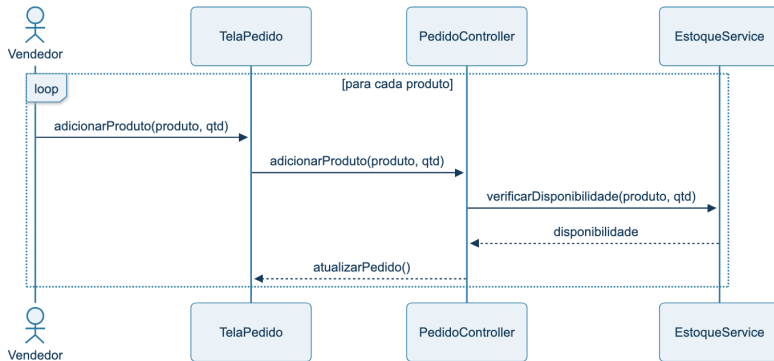
Diagrama:

```
loop [enquanto houver produtos]

    adicionarProduto()
    verificarDisponibilidade()
    adicionarItem()
    recalcularTotal()

end
```

Exemplo com loop





Regra:

0 desconto pode ser informado.

Podemos representar:

```
opt [desconto informado]
    validarDesconto()
    recalcularTotal()
end
```

Se houver dois caminhos relevantes, provavelmente utilizaremos alt.



Temos:

RN03

Desconto > 10% exige autorização.

Isso gera uma interação adicional:

PedidoService



AutorizacaoService



Gerente

ou outro mecanismo definido pela arquitetura.

O importante é:

a decisão arquitetural deve continuar respeitando a regra de negócio.



Evite dois extremos.

Simples demais

```
Usuário → Sistema  
Sistema → Banco  
Banco → Sistema  
Sistema → Usuário
```

Pouco conhecimento de projeto é representado.

Complexo demais

Um diagrama com:

```
30 participantes  
80 mensagens  
12 alternativas
```

pode deixar de comunicar.



Regra prática

Um bom Diagrama de Sequência deve permitir compreender:

- ① qual cenário está sendo realizado;
- ② quem participa;
- ③ qual a responsabilidade de cada participante;
- ④ qual a ordem das interações;
- ⑤ onde estão as decisões relevantes;
- ⑥ qual resultado é produzido.



Passo 1

Escolha um Caso de Uso.

Passo 2

Escolha um cenário específico.

Fluxo Principal
ou
Fluxo Alternativo
ou
Exceção

Passo 3

Liste os passos do cenário.



Passo 4

Identifique os participantes necessários.

```
ator  
interface  
controle  
domínio  
persistência  
serviços externos
```

Passo 5

Transforme cada interação relevante em uma mensagem.

Passo 6

Ordene as mensagens temporalmente.



Passo 7

Identifique:

```
alt  
opt  
loop
```

Passo 8

Verifique as responsabilidades.

Passo 9

Compare novamente com:

```
Caso de Uso  
Requisitos  
Regras de Negócio
```



Checklist — Diagrama de Sequência

- O cenário de origem está identificado?
- O ator está representado?
- Os participantes necessários aparecem?
- Cada participante possui responsabilidade clara?
- As mensagens seguem a ordem do cenário?
- Os nomes das mensagens são compreensíveis?
- Condições importantes aparecem?
- Repetições relevantes aparecem?
- Regras de negócio são respeitadas?
- O resultado final corresponde ao Caso de Uso?



Pergunte:

- A interface contém regra de negócio indevidamente?
- O controller está fazendo tudo?
- O domínio possui comportamento quando apropriado?
- O repositório está sendo usado para persistência?
- Serviços externos estão claramente identificados?
- Existe algum participante sem função real?
- Existe alguma responsabilidade sem responsável?



Faça uma leitura paralela:

CASO DE USO	SEQUÊNCIA
Passo 1	→----- mensagem(ns)
Passo 2	→----- mensagem(ns)
Passo 3	→----- mensagem(ns)
FA01	→----- alt / opt
Repetição	→----- loop
Pós-condição	→----- resultado

Se um passo importante desaparecer:

o diagrama pode estar incompleto.



Testando no sentido inverso

Agora faça:

DIAGRAMA → CASO DE USO

Para cada mensagem importante, pergunte:

De onde surgiu essa responsabilidade?

Se não houver:

- requisito;
- regra;
- passo;
- decisão de projeto justificável;

talvez estejamos **inventando comportamento**.



```
RN03
Desconto > 10% exige autorização
    ↓
RF07
Controlar autorização
    ↓
UC04
Registrar Pedido
    ↓
FA03
Solicitar autorização
    ↓
SEQUÊNCIA
validarDesconto()
solicitarAutorizacao()
    ↓
CÓDIGO
    ↓
TESTE
```

Essa cadeia é extremamente importante em Engenharia de Software.



O que o Diagrama de Sequência NÃO deve expressar?

Ele não é o melhor artefato para mostrar:

- estrutura estática completa do sistema;
- todas as classes e atributos;
- modelo de dados;
- layout da interface;
- cronograma;
- organização das Sprints;
- requisitos em formato textual.

Cada modelo possui uma finalidade.



Uma relação importante:

CASO DE USO

descreve comportamento esperado



DIAGRAMA DE SEQUÊNCIA

descobre/refina colaboração e responsabilidades



DIAGRAMA DE CLASSES

organiza estrutura, relações e responsabilidades

Não significa uma sequência rígida de fases.

Os modelos podem evoluir iterativamente.



Observe:

```
PedidoController → PedidoService:  
registrarPedido()
```

```
PedidoService → EstoqueService:  
verificarDisponibilidade()
```

```
PedidoService → PedidoRepository:  
salvar()
```

Essas mensagens podem indicar operações necessárias na solução.

Posteriormente, isso pode influenciar:

```
interfaces  
classes  
métodos  
serviços
```



O Diagrama pode revelar classes

Se durante a modelagem aparece repetidamente:

Pedido
ItemPedido
Cliente
Produto

podemos investigar:

Esses conceitos devem existir como objetos do domínio?

Assim, a modelagem comportamental ajuda a evoluir para a modelagem estrutural.



Exemplo:

```
TelaPedido  
  ↓  
validarCliente()  
verificarEstoque()  
calcularTotal()  
autorizarDesconto()  
salvarPedido()  
enviarEmail()
```

Pergunta:

A interface deveria possuir todas essas responsabilidades?

Provavelmente existe um problema de distribuição de responsabilidades.



Antes

UC04 - Registrar Pedido

"Usuário registra um pedido."

Depois da análise

RN + RF
↓
Fluxo Principal
↓
Alternativas
↓
Exceções
↓
Critérios
↓
Sequência
↓
Responsabilidades

Agora temos conhecimento suficiente para avançar no projeto.



Atividade — Parte 1

Escolha um Caso de Uso do projeto da equipe.

Aplique o checklist e classifique os problemas encontrados em:

CORREÇÃO
COMPLETUDE
CONSISTÊNCIA
RASTREABILIDADE

Para cada problema:

Problema encontrado:
Por que é um problema:
Correção proposta:



Após revisar o Caso de Uso:

- ① selecione o fluxo principal;
- ② identifique os participantes;
- ③ transforme os passos em mensagens;
- ④ construa o Diagrama de Sequência;
- ⑤ verifique a coerência com RN e RF.



Depois, escolha pelo menos:

```
1 fluxo alternativo  
OU  
1 fluxo de exceção
```

e represente utilizando:

```
alt  
opt  
loop
```

quando apropriado.



A equipe deverá entregar:

1. Caso de Uso revisado.
2. Checklist preenchido.
3. Relação:
RN $\leftrightarrow$ RF/RNF $\leftrightarrow$ UC.
4. Diagrama de Sequência do fluxo principal.
5. Pelo menos uma alternativa/exceção modelada.
6. Breve justificativa das responsabilidades dos participantes.



Perguntas para a revisão entre equipes

Uma equipe deverá conseguir perguntar à outra:

O objetivo está claro?

Alguma regra foi esquecida?

Existe algum caminho sem tratamento?

O diagrama representa o Caso de Uso?

Quem é responsável por cada comportamento?

Existe alguma responsabilidade sem justificativa?

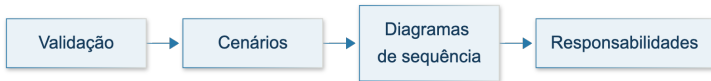
Seria possível implementar e testar isso?



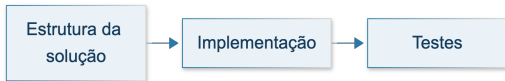
Compreensão do domínio



Validação e interação



Construção e testes





A principal mudança desta aula

Antes:

O sistema deve fazer o quê?

Com Casos de Uso:

Como o ator alcança seu objetivo?

Com Diagramas de Sequência:

Como os participantes colaboram para realizar esse comportamento?

Estamos avançando de:

ANÁLISE DO PROBLEMA



PROJETO DA SOLUÇÃO



Caso de Uso correto

```
objetivo
+ ator
+ contexto
+ fluxo
+ alternativas
+ exceções
+ regras
+ requisitos
+ resultado
```

Diagrama de Sequência correto

```
cenário
+ participantes
+ responsabilidades
+ mensagens
+ ordem temporal
+ decisões
+ resultado
```



Não desenhe o Diagrama de Sequência primeiro.

Faça o caminho:

```
ENTENDER O DOMÍNIO
    ↓
DEFINIR REGRAS
    ↓
ESPECIFICAR REQUISITOS
    ↓
ESPECIFICAR CASO DE USO
    ↓
VALIDAR O CASO DE USO
    ↓
ESCOLHER O CENÁRIO
    ↓
DISTRIBUIR RESPONSABILIDADES
    ↓
MODELAR A SEQUÊNCIA
```

Um bom diagrama é consequência de uma boa compreensão do comportamento.



O Diagrama de Sequência começa a revelar:

OBJETOS
SERVIÇOS
OPERAÇÕES
RESPONSABILIDADES
DEPENDÊNCIAS

A próxima pergunta natural passa a ser:

Como estruturar esses elementos e suas relações?

Essa discussão nos aproxima da:

Modelagem Estrutural / Diagrama de Classes



Referências

- BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. *The Unified Modeling Language User Guide*. Addison-Wesley.
- COCKBURN, A. *Writing Effective Use Cases*. Addison-Wesley.
- FOWLER, M. *UML Distilled*. Addison-Wesley.
- LARMAN, C. *Applying UML and Patterns*. Pearson.
- PRESSMAN, R. S.; MAXIM, B. R. *Software Engineering: A Practitioner's Approach*. McGraw-Hill.
- SOMMERVILLE, I. *Software Engineering*. Pearson.
- OMG. *Unified Modeling Language (UML) Specification*.